

```
select * from customer;
```

---To check for duplicates-----

```
select *,
        row_number() over (partition by order_id order by order_id) as
row_number
from customer;
```

----- handling with duplicates-----

```
with cte as (
        select *,
                row_number() over (partition by order_id order by
order_id) as row_number
from customer)
        select *
        from cte
        where row_number>1;
```

-----checking actual duplicate or not-----

```
with cte as (
        select *,
                row_number() over (partition by order_id order by
order_id) as row_number
from customer)
        select *
```

```
from cte
where order_id in (ord-11121,ord-11135,ord-11158,ord-22254);
```

-----delecting dupliques-----

```
with cte as (
    select *,
           row_number() over (partition by order_id order by
order_id) as row_number
from customer)
select *
from cte
where row_number>1;
```

```
delete from cte
where row_number>1;
```

-----checking null value-----

```
select * from customer
where order_id is null or
order_date is null or
customer_name is null or
customer_segment is null or
country is null or
region is null or
product_category is null or
product_name is null or
quantity is null or
```

unit_price is null or
discount_percent is null or
total_sales is null or
shipping_cost is null or
profit is null or
payment_method is null;

-----second method to find out null value-----

```
select string_agg(  
    'select ''' || column_name || ''' as column_name, count(*) as nullcount ' ||  
    'from customer ' ||  
    'where ' || quote_ident(column_name) || ' is null',  
    ' union all '  
) as dynamic_query  
from information_schema.columns  
where table_name = 'customer';
```

-----update missing value-----

```
select distinct customer_name  
from customer;
```

```
update customer  
    set customer_name = 'unknown'  
    where customer is null;
```

-----we use here mode method to fill customer name if more than 50% data fill that name otherwise no-----

-----we now handle numeric data-----

-----means-----73.32

```
select avg(unit_price)
```

```
from customer;
```

-----mode-----3.89

```
select profit, count(*) as max_count
```

```
from customer
```

```
group by profit
```

```
order by max_count desc;
```

-----median-----46.42

```
select distinct
```

```
percentile_cont(0.5) within group
```

```
(order by unit_price) as median
```

```
from customer;
```

-----filling missing number-----

```
update customer
set unit_price =46.42
where unit_price is null;
```

```
select * from customer;
```

-----Handling negative value-----

```
select * from customer
      where unit_price<0;
```

```
update customer
set unit_price=ABS(unit_price)
where unit_price<0;
```

-----fixing inconsistent date formats and invalid dates-----

```
update customer
set order_date=
      case
      when try_convert(date,order_date,102)
      then try_convert(date,order_date,102)
      else null
end;
```

-----checking the datatype-----

```
select column_name,data_type
from information_schema.columns
where table_name= 'customer';
```

-----change data type-----

```
alter table customer
alter column region type text
using cast (region as text);
```

```
select * from customer;
```

-----identifying and removing outliers-----

```
WITH stats AS (
  SELECT
    AVG(total_sales) AS mean_sales,
    STDDEV(total_sales) AS stddev_sales
  FROM customer
),
flagged_records AS (
  SELECT
    c.*,
    s.mean_sales,
```

```

s.stddev_sales,
CASE
  WHEN ABS(c.total_sales - s.mean_sales) > 3 * s.stddev_sales
  THEN 'Outlier'
  ELSE 'Normal'
END AS outlier_flag
FROM customer c
CROSS JOIN stats s
)

```

----- This returns your customer table WITHOUT the extreme outliers-----

```

SELECT *
FROM flagged_records
WHERE outlier_flag = 'Normal';

```

-----checking affected row before deleting-----

----- Run this first to preview what will be deleted-----

```

SELECT *
FROM customer
WHERE ABS(total_sales - (SELECT AVG(total_sales) FROM customer))
  > 3 * (SELECT STDDEV(total_sales) FROM customer);

```

-----deleting outlier-----

```

DELETE FROM customer
WHERE ABS(total_sales - (SELECT AVG(total_sales) FROM customer))
  > 3 * (SELECT STDDEV(total_sales) FROM customer);

```

-----second syntax-----

```
WITH stats AS (  
  SELECT  
    AVG(total_sales) AS mean_sales,  
    STDDEV(total_sales) AS stddev_sales  
  FROM customer  
)  
DELETE FROM customer c  
USING stats s  
WHERE ABS(c.total_sales - s.mean_sales) > 3 * s.stddev_sales;
```

-----remove unwanted space-----

```
SELECT TRIM(customer_name)  
      FROM customer;
```

----- change into proper case-----

```
SELECT INITCAP(product_name) FROM customer;
```

-----second method-----

```
UPDATE customer  
SET product_name = INITCAP(product_name);
```



```
select * from customer;
```